Patient Registration Implementation Plan

Saranam Home Care Platform

Prepared: 30 March 2026

Based on: Client Meeting - 30 March 2026

Scope of Changes

1. Patient Category: Boolean (BSC/N-BSC) --> 4-value enum system
2. Diagnosis: New mandatory dropdown field (18 cancer types)
3. Referred From: New field with predefined referral sources
4. Patient Priority: Allow blank at registration
5. Date of Birth: Approximate age support
6. Geocoding: Search-based location lookup

1. Current State Analysis

1.1 Patient Category - Current Implementation

The patient category is currently stored as a BOOLEAN field 'bsc_status' in the patients table. True = BSC (Best Supportive Care), False = N-BSC. This boolean is then mirrored as a string 'care_status' in the patient_scheduling_meta table ('bsc' or 'n_bsc').

Files that directly reference bsc_status / care_status:

Layer	File	Usage
Backend	patients/models.py	Boolean column definition
Backend	patients/schemas.py	Pydantic validation (5 schemas)
Backend	patients/routes.py	API filter parameter
Backend	patients/services.py	Response builder + search filter
Backend	saranam/models/patient_scheduling_meta.py	care_status string field
Backend	saranam/services/patient_pool_service.py	Priority scoring + pool filtering
Backend	saranam/services/schedule_service.py	BSC statistics calculation
Backend	call_logs/service.py	Call priority ordering
Backend	seed_demo_data.py	Demo data seeding
Frontend	components/patients/PatientQuickAdd.jsx	Registration form toggle
Frontend	pages/patients/PatientsList.jsx	Filter: All/BSC/N-BSC
Frontend	api/patients.js	API param: bsc_status
Frontend	context/PatientContext.jsx	State mgmt: bscStatus param
Frontend	pages/saranam/scheduling/PatientPoolPage.jsx	Care status filter
Frontend	pages/saranam/scheduling/WeeklyPlanningBoard.jsx	BSC display/badges
Frontend	pages/saranam/scheduling/KitReadinessPage.jsx	BSC reference
Frontend	pages/saranam/scheduling/AlertsPage.jsx	BSC alerts
Frontend	pages/saranam/scheduling/MyVisitsPage.jsx	BSC display
Frontend	pages/saranam/scheduling/ReviewDashboardPage.jsx	BSC review stats
Frontend	pages/saranam/scheduling/DailyVisitPage.jsx	BSC display
Frontend	utils/generateSchedulePDF.js	PDF: BSC column + stats

!! WARNING

CRITICAL: Changing bsc_status from Boolean to a 4-value string field is a cross-cutting change that touches 21+ files. A database migration is required, and all existing data must be mapped. This is the highest-risk change in the patient registration scope.

1.2 Priority Scoring Algorithm - Current Logic

The scheduling priority score currently uses bsc_status as a major factor:

```
if meta.care_status == 'bsc':      score += 50   # BSC gets +50
complexity_scores = {'high': 30, 'medium': 20, 'low': 10}
if meta.last_visit_date is None:    score += 25   # Never visited
if meta.lead_flagged:               score += 40   # Flagged by lead
if overdue_days > 0:               score += overdue_days * 5
```

With 4 categories, the scoring needs to be re-calibrated. Proposed new scoring is covered in Section 3 (Phase 1, Task 6).

2. Questions Requiring Clarification

> **!! WARNING**
> These questions MUST be answered before implementation begins. Answers will determine migration strategy and algorithm changes.

Q1: Data Migration - Existing Patients

Current patients have bsc_status = true (BSC) or false (N-BSC). When we migrate to 4 categories, we don't know if an existing BSC patient is Onco or Non-Onco.

Options:
A) Map all existing BSC=true --> 'BSC-Onco' and BSC=false --> 'Non-BSC-Onco' (assume all are Onco)
B) Map all to a temporary 'Unclassified-BSC' / 'Unclassified-Non-BSC' and require manual reclassification
C) Map based on diagnosis data if available, otherwise default to Onco

Recommendation: Option A (simplest, can be corrected later)

Q2: Priority Scoring for New Categories

Currently BSC patients get +50 priority points. With 4 categories, what should the hierarchy be?

Proposed scoring:
BSC-Onco: +50 (highest priority - palliative oncology)
BSC-Non-Onco: +40 (palliative non-oncology)
Non-BSC-Onco: +30 (active treatment oncology)
Non-BSC-Non-Onco: +20 (active treatment non-oncology)

Does this hierarchy match clinical priority? Please confirm or adjust.

Q3: Scheduling Pool Filters

Currently the patient pool can be filtered by 'bsc' or 'n_bsc'. With 4 categories:

Option A) Show all 4 as individual filter options
Option B) Group filters: 'All BSC' / 'All Non-BSC' / 'All Onco' / 'All Non-Onco' + individual 4
Option C) Two separate filter dropdowns: one for BSC/Non-BSC, one for Onco/Non-Onco

Recommendation: Option C (gives most flexibility with least UI clutter)

Q4: Diagnosis Dropdown - Editable by Admin?

The client said 'we can add to this, if the access is given'. Should we:

A) Hardcode the 18 cancer types + 'Other' in frontend (simpler, faster)
B) Create a diagnosis_types table in DB with an admin UI to manage entries

Recommendation: Option A for now, migrate to B in a later sprint

Q5: Referred From - Values

Confirmed sources: KC General, Kidwai, Doctor Outside.
Should we also include: Self-referral, NGO referral, PHC referral?

Should this be a free-text + dropdown combo or strict dropdown only?

Q6: Approximate Age - How to store?

When a patient gives approximate age (e.g., '50'), we calculate DOB as Jan 1, (current_year - age). Should we add an 'is_approximate_dob' boolean flag so the system knows the DOB was estimated? This matters for age-based analytics accuracy.

3. Phase 1 - Patient Category System (Boolean --> Enum)

This is the highest-priority and highest-risk change. It must be done first because all other features (diagnosis display, PDF changes, scheduling) depend on the category system.

Step 1: Database Migration

1a. Create new column 'patient_category' (String(20)) in patients table

`backend/app/patients/models.py`

> Add column: patient_category = Column(String(20), nullable=False, default='BSC_ONCO')

> Valid values: 'BSC_ONCO', 'NON_BSC_ONCO', 'BSC_NON_ONCO', 'NON_BSC_NON_ONCO'

1b. Write Alembic migration script

`backend/migrations/versions/xxxx_patient_category_enum.py`

> Add patient_category column with server_default='BSC_ONCO'

> Migrate existing data: bsc_status=True --> 'BSC_ONCO', bsc_status=False --> 'NON_BSC_ONCO'

> Keep bsc_status column temporarily for rollback safety

> Add index on patient_category for query performance

```
# Migration pseudo-code:
op.add_column('patients', sa.Column('patient_category', sa.String(20),
              server_default='BSC_ONCO', nullable=False))
op.execute("UPDATE patients SET patient_category = "
           "CASE WHEN bsc_status = true THEN 'BSC_ONCO' "
           "ELSE 'NON_BSC_ONCO' END")
op.create_index('ix_patients_patient_category', 'patients', ['patient_category'])
```

1c. Update PatientSchedulingMeta.care_status to match new values

`backend/app/saranam/models/patient_scheduling_meta.py`

> Change care_status from String(10) to String(20)

> Migrate: 'bsc' --> 'BSC_ONCO', 'n_bsc' --> 'NON_BSC_ONCO'

!! WARNING

DO NOT drop the bsc_status column in this migration. Keep it for 1 sprint as a rollback safety net. Drop it in a follow-up migration after confirming everything works.

Step 2: Backend Model & Schema Updates

2a. Update Patient model - add patient_category, deprecate bsc_status

`backend/app/patients/models.py`

> Add: patient_category = Column(String(20), nullable=False, default='BSC_ONCO')

> Add helper property: is_bsc --> returns True if category starts with 'BSC_'

> Add helper property: is_onco --> returns True if category ends with '_ONCO'

2b. Update ALL Pydantic schemas (5 schemas)

`backend/app/patients/schemas.py`

> PatientCreateSchema: Replace bsc_status:bool with patient_category:str + validator

> PatientUpdateSchema: Replace bsc_status:Optional[bool] with patient_category:Optional[str]

> PatientResponseSchema: Replace bsc_status:bool with patient_category:str

> PatientListItemSchema: Replace bsc_status:bool with patient_category:str

> PatientSearchSchema: Replace bsc_status:Optional[bool] with patient_category:Optional[str]

> Add validator: validate_patient_category with whitelist of 4 values

```
VALID_CATEGORIES = ['BSC_ONCO', 'NON_BSC_ONCO', 'BSC_NON_ONCO', 'NON_BSC_NON_ONCO']

CATEGORY_LABELS = {
    'BSC_ONCO': 'BSC Onco',
    'NON_BSC_ONCO': 'Non-BSC Onco',
    'BSC_NON_ONCO': 'BSC Non-Onco',
    'NON_BSC_NON_ONCO': 'Non-BSC Non-Onco',
}
```

Step 3: Backend Service & Route Updates

3a. Update patient service layer

`backend/app/patients/services.py`

> _prepare_patient_response: Return patient_category instead of bsc_status

> search_patients: Update filter logic from boolean to string matching

> Support filtering by: exact category, 'all_bsc' (both BSC_*), 'all_onco' (both *_ONCO)

3b. Update patient routes

`backend/app/patients/routes.py`

> Change query param from 'bsc_status' (bool) to 'patient_category' (string)

> Support comma-separated values for multi-filter: ?patient_category=BSC_ONCO,BSC_NON_ONCO

3c. Update call log service

`backend/app/call_logs/service.py`

> Change: desc(Patient.bsc_status) --> custom ordering by patient_category

> Order: BSC_ONCO > BSC_NON_ONCO > NON_BSC_ONCO > NON_BSC_NON_ONCO

Step 4: Scheduling System Updates

4a. Update patient pool service - priority scoring

`backend/app/saranam/services/patient_pool_service.py`

> Replace: if care_status == 'bsc': score += 50

> With tiered scoring:

```
CATEGORY_SCORES = {
    'BSC_ONCO': 50,
    'BSC_NON_ONCO': 40,
    'NON_BSC_ONCO': 30,
    'NON_BSC_NON_ONCO': 20,
}
score += CATEGORY_SCORES.get(meta.care_status, 20)
```

4b. Update patient pool filtering

`backend/app/saranam/services/patient_pool_service.py`

> Replace boolean filter with string-based category filter

> Update _ensure_meta_for_patient to map patient_category --> care_status

4c. Update schedule service statistics

`backend/app/saranam/services/schedule_service.py`

> Replace: bsc_patients count --> category breakdown dict

> Track: {BSC_ONCO: 3, NON_BSC_ONCO: 2, ...} instead of single bsc count

Step 5: Frontend Updates

5a. Update PatientQuickAdd registration form

`frontend/src/components/patients/PatientQuickAdd.jsx`

> Replace BSC toggle (boolean) with a 4-option dropdown/radio group
> Options: BSC Onco, Non-BSC Onco, BSC Non-Onco, Non-BSC Non-Onco
> Default: none selected (force user to choose)
> Update form state: bsc_status:bool --> patient_category:string
> Update submission payload

5b. Update PatientsList page filters

`frontend/src/pages/patients/PatientsList.jsx`

> Replace 3-state toggle (All/BSC/N-BSC) with dropdown
> Options: All, BSC Onco, Non-BSC Onco, BSC Non-Onco, Non-BSC Non-Onco
> Update badge colors: 4 distinct colors for 4 categories

5c. Update API client

`frontend/src/api/patients.js`

> Change param name: bsc_status --> patient_category
> Update search() function signature and callers

5d. Update PatientContext

`frontend/src/context/PatientContext.jsx`

> Change: bscStatus param --> patientCategory in searchPatients()

5e. Update ALL scheduling pages (6 files)

`frontend/src/pages/saranam/scheduling/*.jsx`

> PatientPoolPage.jsx - Update CARE_STATUS_OPTIONS from ['bsc','n_bsc'] to 4 values
> WeeklyPlanningBoard.jsx - Update category badges and display
> KitReadinessPage.jsx - Update BSC references
> AlertsPage.jsx - Update BSC alert logic
> MyVisitsPage.jsx - Update category display
> ReviewDashboardPage.jsx - Update review stats
> DailyVisitPage.jsx - Update category display

5f. Update Schedule PDF generation

`frontend/src/utils/generateSchedulePDF.js`

> Replace 'BSC' column with 'Category' column
> Show abbreviated labels: BSC-O, NB-O, BSC-NO, NB-NO
> Update statistics section: show category breakdown instead of BSC count
> Update cell coloring logic for 4 categories

5g. Update seed demo data

`backend/seed_demo_data.py`

> Replace 'bsc': True/False with 'category': 'BSC_ONCO' etc.
> Distribute demo patients across all 4 categories

Step 6: Create Shared Constants

To avoid duplication and ensure consistency, create shared constant files:

6a. Backend constants

`backend/app/constants.py (new file)`

```
PATIENT_CATEGORIES = ['BSC_ONCO','NON_BSC_ONCO','BSC_NON_ONCO','NON_BSC_NON_ONCO']
CATEGORY_LABELS = {...}
CATEGORY_PRIORITY_SCORES = {...}
```

6b. Frontend constants

`frontend/src/constants/patientCategories.js (new file)`

```
export const PATIENT_CATEGORIES = [
  { value: 'BSC_ONCO', label: 'BSC Onco', short: 'BSC-O', color: '#EF4444' },
  { value: 'NON_BSC_ONCO', label: 'Non-BSC Onco', short: 'NB-O', color: '#F59E0B' },
  { value: 'BSC_NON_ONCO', label: 'BSC Non-Onco', short: 'BSC-NO', color: '#3B82F6' },
  { value: 'NON_BSC_NON_ONCO', label: 'Non-BSC Non-Onco', short:'NB-NO', color:'#10B981'},
];
```

4. Phase 2 - New Registration Fields

4.1 Diagnosis Field (Mandatory Dropdown)

1. **Add diagnosis column to patients table**
   `backend/app/patients/models.py`

> diagnosis = Column(String(100), nullable=True) # nullable for existing data

> Make mandatory in schema validation (PatientCreateSchema) but nullable in DB for migration

2. **Write migration: add diagnosis column**
   `backend/migrations/versions/xxxx_add_diagnosis_field.py`

> Add column with nullable=True (existing patients won't have diagnosis)

> Set default=NULL for existing records

3. **Update schemas with diagnosis field + validator**
   `backend/app/patients/schemas.py`

> PatientCreateSchema: diagnosis: str (required, validated against dropdown list)

> PatientUpdateSchema: diagnosis: Optional[str]

> PatientResponseSchema, PatientListItemSchema: include diagnosis

4. **Add diagnosis dropdown to PatientQuickAdd form**
   `frontend/src/components/patients/PatientQuickAdd.jsx`

> Searchable dropdown with 18 cancer types + 'Other' with free-text

> Mark as required (red asterisk, validation)

5. **Display diagnosis in PatientsList and PatientCard**
   `frontend/src/pages/patients/PatientsList.jsx`

> Add diagnosis badge/tag to patient list items

> Add diagnosis filter dropdown to list filters

```
DIAGNOSIS_OPTIONS = [
  'Breast cancer', 'Oral cancer', 'Cervical cancer', 'Lung cancer',
  'Colorectal cancers', 'Stomach cancer', 'GEJ carcinoma', 'ALL',
  'Neuroblastoma', 'Ewings sarcoma', 'Renal Cell Carcinoma',
  'Prostate cancer', 'Melanoma', 'Liver cancer', 'Gallbladder cancer',
  'Bladder cancer', 'Penile cancer', 'Pancreatic cancer', 'Other'
]
```

4.2 Referred From (New Field)

1. **Add referred_from column to patients table**
   `backend/app/patients/models.py`

> referred_from = Column(String(100), nullable=True)

2. **Write migration + update schemas**
   `backend/app/patients/schemas.py`

> Optional field in create/update schemas

> Validator against predefined list + 'Other' option

3. **Add dropdown in PatientQuickAdd**
   `frontend/src/components/patients/PatientQuickAdd.jsx`

> Options: KC General Hospital, Kidwai Memorial Institute, Doctor Outside, Self-Referral, Other

> 'Other' shows a text input for custom entry

4.3 Patient Priority (Allow Blank)

clinical_complexity currently defaults to 'medium'. Change to allow NULL/blank at registration.

1. **Update schema default from 'medium' to None**
   `backend/app/patients/schemas.py`
2. **Update model default from 'medium' to None**
   `backend/app/patients/models.py`
3. **Update frontend to show 'Not Assessed' when blank**
   `frontend/src/components/patients/PatientQuickAdd.jsx`

> Remove default selection in complexity dropdown

> Add placeholder: 'Will be assessed after first visit'

4.4 Date of Birth - Approximate Age Support

1. **Add is_approximate_dob flag to Patient model**
   `backend/app/patients/models.py`

> is_approximate_dob = Column(Boolean, default=False)

2. **Update PatientQuickAdd form**
   `frontend/src/components/patients/PatientQuickAdd.jsx`

> Add toggle: 'Exact DOB' / 'Approximate Age'

> When 'Approximate Age' selected: show number input for age

> Auto-calculate: DOB = Jan 1, (current_year - entered_age)

> Set is_approximate_dob = true in submission

3. **Display indicator in patient views**
   `frontend/src/pages/patients/PatientDetail.jsx`

> Show '~' prefix before age when is_approximate_dob is true (e.g., '~50 yrs')

4.5 Geocoding Search Enhancement

1. **Integrate geocoding API in map component**
   `frontend/src/components/patients/PatientQuickAdd.jsx`

> Add search input above map

> Use Google Maps Geocoding or OpenStreetMap Nominatim API

> On search: move map pin to result location, auto-fill address fields

2. **Show distance from Kidwai after location capture**

> Calculate haversine distance from patient location to Kidwai coordinates

> Display: 'Approximately X.X km from Kidwai Memorial Hospital'

> Show within/outside service radius indicator

5. Implementation Checklist & Order

Execute in this exact order. Phase 1 (Category) must be fully complete before Phase 2 because diagnosis display, PDF generation, and scheduling all depend on the new category system.

Phase 1: Patient Category (Estimated: 2-3 days)

#	Task	Est.
1	Create shared constants (backend + frontend)	0.5h
2	Write Alembic migration for patient_category + care_status	1h
3	Update Patient model (add patient_category, keep bsc_status)	0.5h
4	Update all 5 Pydantic schemas + validators	1h
5	Update patient services (response, search, filters)	1h
6	Update patient routes (API param change)	0.5h
7	Update patient_pool_service (priority scoring + filtering)	1.5h
8	Update schedule_service (statistics)	0.5h
9	Update call_logs service (ordering)	0.5h
10	Update PatientQuickAdd form (toggle --> dropdown)	1h
11	Update PatientsList (filters + badges)	1h
12	Update API client + PatientContext	0.5h
13	Update 6 scheduling pages	2h
14	Update generateSchedulePDF.js	1h
15	Update seed_demo_data.py	0.5h
16	End-to-end testing	2h

Phase 2: New Fields (Estimated: 1-2 days)

#	Task	Est.
1	Migration: add diagnosis, referred_from, is_approximate_dob columns	0.5h
2	Update Patient model with new columns	0.5h
3	Update schemas + validators for new fields	1h
4	Update patient service response builder	0.5h
5	Frontend: Diagnosis dropdown in PatientQuickAdd	1h
6	Frontend: Referred From dropdown in PatientQuickAdd	0.5h
7	Frontend: Approximate age toggle + auto-DOB calculation	1h
8	Frontend: Geocoding search in map component	1.5h
9	Frontend: Distance from Kidwai display	1h
10	Frontend: clinical_complexity allow blank + 'Not Assessed' display	0.5h
11	Display new fields in PatientsList, PatientDetail, PatientCard	1h

12	End-to-end testing	1.5h

6. Risk Assessment

Risk	Description	Mitigation
HIGH	Category migration breaks scheduling	Run migration on staging first. Keep bsc_status for rollback.
HIGH	Priority scoring imbalance	Test with demo data. Get clinical team sign-off on scores.
MED	Frontend filter state inconsistency	Clear all cached filters on deploy. Version API responses.
MED	PDF generation breaks	Snapshot current PDFs. Compare before/after visually.
LOW	Existing API consumers break	Document API changes. Provide 1-sprint deprecation period.
LOW	Geocoding API rate limits	Cache results. Use free tier Nominatim with fallback.

7. Complete File Impact Map

Backend Files (11 files)

> app/patients/models.py -- Add patient_category, diagnosis, referred_from, is_approximate_dob
> app/patients/schemas.py -- Update 5 schemas, add new field validators
> app/patients/routes.py -- Change filter params
> app/patients/services.py -- Update response builder, search filters
> app/saranam/models/patient_scheduling_meta.py -- Widen care_status to String(20)
> app/saranam/services/patient_pool_service.py -- New priority scoring, new filters
> app/saranam/services/schedule_service.py -- Category breakdown stats
> app/call_logs/service.py -- Update ordering
> app/constants.py -- NEW: shared constants
> migrations/versions/xxxx_*.py -- NEW: 2 migration files
> seed_demo_data.py -- Update demo data

Frontend Files (13 files)

> src/constants/patientCategories.js -- NEW: shared constants + colors
> src/components/patients/PatientQuickAdd.jsx -- Category dropdown, diagnosis, referred_from, DOB toggle
> src/pages/patients/PatientsList.jsx -- Filter updates, badge colors
> src/pages/patients/PatientDetail.jsx -- Display new fields
> src/components/patients/PatientCard.jsx -- Category badge update
> src/api/patients.js -- API param rename
> src/context/PatientContext.jsx -- State param rename
> src/pages/saranam/scheduling/PatientPoolPage.jsx -- Category filter
> src/pages/saranam/scheduling/WeeklyPlanningBoard.jsx -- Category display
> src/pages/saranam/scheduling/KitReadinessPage.jsx -- Category reference
> src/pages/saranam/scheduling/AlertsPage.jsx -- Category alerts
> src/pages/saranam/scheduling/MyVisitsPage.jsx -- Category display
> src/pages/saranam/scheduling/ReviewDashboardPage.jsx -- Category stats
> src/pages/saranam/scheduling/DailyVisitPage.jsx -- Category display

> src/utils/generateSchedulePDF.js -- PDF category column

Total files impacted: 24 files (11 backend + 13 frontend) + 2 new files